University of Calgary
CPSC 433 – Artificial Intelligence

Course Timetabling via Three Search Paradigms

OR-Tree Feasibility, Set-Based Improvement, and AND-Tree Optimization

Formal Search Models and Processes – Proposal

Member 1: Syed Essam Uddin Khawaja 30242943

Member 2: Nova Driscoll 30211940

Member 3: [NAME / UCID]

Member 4: [NAME / UCID]

Member 5: [NAME / UCID]

Date: [SUBMISSION DATE]

Contents

1 Introduction **3**

2 Formal Scheduling Problem **3**

2.1 Components, slots, and domains . . . 3

2.2 Hard constraints . . . 5

2.3 Soft constraints and objective . . . 6

3 Overall Search Architecture **8**

3.1 How the models relate . . . 8

4 Model I: OR-Tree Feasibility Search **8**

4.1 Search model: $A_\vee = (S_\vee, T_\vee)$ . . . 8

4.2 Search process: $P_\vee = (A_\vee, Env_\vee, K_\vee)$ . . . 9

4.3 Search instance: $Inst_\vee = (q_0, G_\vee)$ . . . 9

4.4 Algorithm 1: OR-tree feasibility . . . 10

4.5 Worked OR-tree derivation . . . 10

5 Model II: Set-Based Local Optimization **10**

5.1 Search model: $A_{set} = (S_{set}, T_{set})$ . . . 11

5.2 Search process: $P_{set} = (A_{set}, Env_{set}, K_{set})$ . . . 13

5.3 Search instance: $Inst_{set} = (q_0, G_{set})$ . . . 15

5.4 Algorithm 2: Set-based hill climbing . . . 16

5.5 Worked set-based derivation . . . 17

5.6 Justification and limitations . . . 20

6 Solution 2: AND-Tree Optimization Following OR-Tree Feasibility **21**

6.1 Purpose and connection to the feasibility stage . . . 21

6.2 Scheduling notation and shared functions . . . 21

6.3 Search model . . . 22

6.4 Search process . . . 24

6.5 Search instance and termination . . . 25

6.6 Operational procedure . . . 25

6.7 Correctness and limitations . . . 26

6.8 Demonstration on a small search instance . . . 28

7 End-to-End Integration and Comparison **29**

7.1 Algorithm 4: Optional combined pipeline 29

7.2 Comparison of the three paradigms 30

7.3 Unified worked example 30

8 Discussion and Limitations **30**

9 Conclusion **30**

A Complete Constraint Formulas **31**

B Additional Derivations and Search Trees **31**

Abstract

[**TO FILL:** Write 150–200 words: problem, all three paradigms, different state representations, search controls, and the benefit/limitation of each. Write this last.]

1 Introduction

We are developing a system to make academic schedules for a semester at the University of Calgary, a single weekly schedule (which is then repeated for 13 weeks) containing *Lectures* and *Tutorials* sections (belonging to a course) which fit in weekly room time *Slots* with designated start and end times. There are two categories of room time *Slots*, one for lectures and the other for tutorials, along with specific "active learning" designation for some rooms, with the number of rooms limited per slot. The objective is to assign lectures and tutorials to permitted weekly slots so that every hard constraint holds and the soft-constraint penalty is minimized. This problem is similar to what a student would do when creating a weekly class schedule in Schedule Builder, although we are creating the section times which students pick from. There is no direct step-based solution to this problem; therefore a procedural approach would require brute force. The number of possible timetables grows exponentially, about $|\mathit{Slots}|^{|X|}$. Search scales better in practice as it abandons a partial timetable as soon as it breaks a hard constraint.

This proposal compares two ways to improve a valid timetable: OR-tree feasibility followed by set-based local search, or OR-tree feasibility followed by AND-tree branch-and-bound. Section 2 formalizes the scheduling instance once; later sections instantiate the search models and controls without redefining the same constraints.

2 Formal Scheduling Problem

We use the terminology and symbols of the problem description [2] throughout. Symbols it does not define, such as *overlap*, $D(x)$ and Consistent, are introduced below.

2.1 Components, slots, and domains

Components: *Lectures* is the set of lecture sections and *Tutorials* the set of tutorials and labs; labs (`LAB`) are treated exactly like tutorials (`TUT`). The set of all components to schedule is $X = \mathit{Lectures} + \mathit{Tutorials}$. Each lecture is named *program, course number, section*, for example `CPSC 433 LEC 01`. For $x \in X$, $\mathit{course}(x)$ is the pair (program, number), e.g. (`CPSC`, 433).

Which lectures a tutorial belongs to: A tutorial must not clash with its lecture. The tutorial's name tells us which lecture that is, and there are two naming forms:

- `CPSC 433 LEC 01 TUT 02` names a section. It belongs to `CPSC 433 LEC 01` only.
- `CPSC 433 TUT 01` names no section. It is a shared tutorial, open to students of every section, so it belongs to all lectures of `CPSC 433`.

The set of lectures that tutorial t belongs to is

$$\mathit{Lec}(t) = \begin{cases} \{\text{the lecture named in } t\} & \text{if } t\text{'s name has a } \texttt{LEC} \text{ part,} \\ \{\, l \in \mathit{Lectures} : \mathit{course}(l) = \mathit{course}(t) \,\} & \text{otherwise (shared tutorial).} \end{cases} \tag{1}$$

For example, if the input has `CPSC 433 LEC 01` and `CPSC 433 LEC 02`, then

$$Lec(\texttt{CPSC 433 LEC 02 TUT 01}) = \{\texttt{CPSC 433 LEC 02}\},$$
$$Lec(\texttt{CPSC 433 TUT 01}) = \{\texttt{CPSC 433 LEC 01}, \texttt{CPSC 433 LEC 02}\}.$$

Weekly slots: The UofC rules link days together. A lecture placed on Monday must also meet Wednesday and Friday at the same time; anything placed on Tuesday must also meet Thursday; a tutorial placed on Monday must also meet Wednesday. So the scheduler never picks single days. It picks a *day pattern and a start time*, and the linked days follow automatically. We call one such choice a *slot*. Because the linking is built into the slot, the day-linking rules always hold and do not appear among the hard constraints in Section 2.2.

Formally, a slot is a triple

$$s = (days(s),\ start(s),\ end(s)), \tag{2}$$

meaning "meet on every day in $days(s)$, from $start(s)$ to $end(s)$." For example, the lecture slot used for a Monday 9:00 lecture is $(\{\text{Mon}, \text{Wed}, \text{Fri}\},\ 9{:}00,\ 10{:}00)$. For short, we name a slot by its day pattern and start time, so this slot is written *MWF 9:00*. The day patterns are written MWF (Mon/Wed/Fri), TT (Tue/Thu), MW (Mon/Wed) and F (Fri only).

Lectures and tutorials use different slot patterns; Table 1 lists them all.

Table 1: Slot patterns. Each row is one group of slots; h is the start time.

Used by	Written as	Meets on	Length	Start times	Count
Lectures	MWF h	Mon, Wed, Fri	1 h	8:00, 9:00, …, 20:00	13
Lectures	TT h	Tue, Thu	1.5 h	8:00, 9:30, …, 18:30	8
Tutorials	MW h	Mon, Wed	1 h	8:00, 9:00, …, 20:00	13
Tutorials	TT h	Tue, Thu	1 h	8:00, 9:00, …, 20:00	13
Tutorials	F h	Fri only	2 h	8:00, 10:00, …, 18:00	6

A given input offers only some of these slots, each with its capacities. *Slots* is the set of offered slots. The offered lecture slots (top half of Table 1) form $Slots_L$ and the offered tutorial slots (bottom half) form $Slots_T$, so $Slots = Slots_L + Slots_T$. Lectures may only use $Slots_L$ and tutorials only $Slots_T$.

Time overlap: Since lecture and tutorial slots follow different grids, we compare actual meeting times. Two slots s and s' *overlap* when they meet on a common day and their time ranges intersect:

$$overlap(s, s') \iff days(s) \cap days(s') \neq \varnothing \ \wedge\ start(s) < end(s') \ \wedge\ start(s') < end(s). \tag{3}$$

For example, lecture slot TT 9:30 (9:30-11:00) overlaps tutorial slots TT 9:00 and TT 10:00.

Two lectures, or two tutorials, can only clash if they are in the same slot, because slots of the same kind either fall on different days or follow each other without overlapping:

$$s, s' \in Slots_L \ \text{ or } \ s, s' \in Slots_T \implies \big(overlap(s, s') \iff s = s'\big). \tag{4}$$

Domains: The *domain* $D(x)$ is the set of slots component x may use: the slots of its kind ($Slots_L$ or $Slots_T$), minus any slot ruled out by a hard constraint on x alone (C_5, C_8 to C_{10} and C_{12} in Section 2.2).

Assignments: A complete timetable is a function

$$\mathit{assign} : \mathit{Lectures} + \mathit{Tutorials} \to \mathit{Slots}, \qquad \mathit{assign}(x) \in D(x) \text{ for every } x \in X. \tag{5}$$

While a timetable is being built, only some components have a slot yet. Such a *partial assignment* *assign* is defined only on the components $\mathrm{dom}(\mathit{assign}) \subseteq X$ placed so far. (This is different from the input preassignment *partassign* in Section 2.2.)

Slot counts: The capacity constraints (C_1 to C_4) need to know how full each slot is. For an assignment *assign* and a slot s, $n_L(s)$ is the number of lectures *assign* puts in s and $n_T(s)$ the number of tutorials. $n_{L,AL}(s)$ and $n_{T,AL}(s)$ count only the ones with an *activelearning* statement. Only placed components are counted, so the same counts work for partial assignments:

$$\begin{aligned}
n_L(s) &= |\{l \in \mathit{Lectures} \cap \mathrm{dom}(\mathit{assign}) : \mathit{assign}(l) = s\}|,\\
n_T(s) &= |\{t \in \mathit{Tutorials} \cap \mathrm{dom}(\mathit{assign}) : \mathit{assign}(t) = s\}|,\\
n_{L,AL}(s) &= |\{l \in \mathit{Lectures} \cap \mathrm{dom}(\mathit{assign}) : \mathit{activelearning}(l),\ \mathit{assign}(l) = s\}|,\\
n_{T,AL}(s) &= |\{t \in \mathit{Tutorials} \cap \mathrm{dom}(\mathit{assign}) : \mathit{activelearning}(t),\ \mathit{assign}(t) = s\}|.
\end{aligned}$$

For example, if *assign* puts `CPSC 433 LEC 01` and `DATA 201 LEC 01` in MWF 9:00 and only `DATA 201 LEC 01` needs active learning, then $n_L(\text{MWF 9:00}) = 2$ and $n_{L,AL}(\text{MWF 9:00}) = 1$. C_1 then checks $2 \le \mathit{lecturemax}(\text{MWF 9:00})$ and C_3 checks $1 \le \mathit{ALlecturemax}(\text{MWF 9:00})$.

2.2 Hard constraints

Hard constraints are the rules a valid timetable cannot break. Besides the sets defined above, they use these inputs:

- lecture capacities $\mathit{lecturemax}(s)$ and $\mathit{ALlecturemax}(s)$ for each $s \in \mathit{Slots}_L$;
- tutorial capacities $\mathit{tutorialmax}(s)$ and $\mathit{ALtutorialmax}(s)$ for each $s \in \mathit{Slots}_T$;
- the $\mathit{notcompatible}(x, y)$ and $\mathit{unwanted}(x, s)$ statements;
- the preassignment $\mathit{partassign} : \mathit{Lectures} + \mathit{Tutorials} \to \mathit{Slots} + \{\$\}$, where \$ means no preassignment;
- the $\mathit{activelearning}(x)$ statements.

Here $\mathit{ALmax}(s)$ means $\mathit{ALlecturemax}(s)$ if $s \in \mathit{Slots}_L$ and $\mathit{ALtutorialmax}(s)$ if $s \in \mathit{Slots}_T$. Each constraint only looks at components that have been placed, so it can also be checked on a partial

assignment.

$$C_1:\ \forall s \in \mathit{Slots}_L:\ n_L(s) \le \mathit{lecturemax}(s) \quad \text{(lecture capacity)} \quad (6)$$
$$C_2:\ \forall s \in \mathit{Slots}_T:\ n_T(s) \le \mathit{tutorialmax}(s) \quad \text{(tutorial capacity)} \quad (7)$$
$$C_3:\ \forall s \in \mathit{Slots}_L:\ n_{L,AL}(s) \le \mathit{ALlecturemax}(s) \quad \text{(AL lecture capacity)} \quad (8)$$
$$C_4:\ \forall s \in \mathit{Slots}_T:\ n_{T,AL}(s) \le \mathit{ALtutorialmax}(s) \quad \text{(AL tutorial capacity)} \quad (9)$$
$$C_5:\ \forall x \text{ with } \mathit{activelearning}(x):\ \mathit{ALmax}(\mathit{assign}(x)) > 0 \quad \text{(AL placement)} \quad (10)$$
$$C_6:\ \forall t \in \mathit{Tutorials},\ \forall l \in \mathit{Lec}(t):\ \neg\, \mathit{overlap}(\mathit{assign}(l), \mathit{assign}(t)) \quad \text{(lecture/tutorial)} \quad (11)$$
$$C_7:\ \forall\, \mathit{notcompatible}(x, y):\ \neg\, \mathit{overlap}(\mathit{assign}(x), \mathit{assign}(y)) \quad \text{(not compatible)} \quad (12)$$
$$C_8:\ \forall x \text{ with } \mathit{partassign}(x) \neq \$:\ \mathit{assign}(x) = \mathit{partassign}(x) \quad \text{(preassignment)} \quad (13)$$
$$C_9:\ \forall\, \mathit{unwanted}(x, s):\ \mathit{assign}(x) \neq s \quad \text{(unwanted)} \quad (14)$$
$$C_{10}:\ \forall x \in X_9:\ \mathit{start}(\mathit{assign}(x)) \ge 18{:}00 \quad \text{(LEC 9 evening)} \quad (15)$$
$$C_{11}:\ \forall l, l' \in L_5 \text{ with } l \neq l':\ \mathit{assign}(l) \neq \mathit{assign}(l') \quad \text{(5XX non-overlap)} \quad (16)$$
$$C_{12}:\ \forall l \in \mathit{Lectures}:\ \mathit{assign}(l) \neq \text{TT } 11{:}00 \quad \text{(TT 11:00 meeting)} \quad (17)$$
$$C_{13}:\ \text{special CPSC 851/913 bookings (below)}$$

X_9 is the set of components whose name has a `LEC` number starting with 9, such as `CPSC 433 LEC 90` and `CPSC 433 LEC 90 TUT 01`; a shared tutorial such as `CPSC 433 TUT 01` has no `LEC` number and is not in X_9. L_5 is the set of 5XX lectures across all programs; because two lecture slots overlap only if they are equal (4), C_{11} keeps them from overlapping. The problem description writes C_6 as $\mathit{assign}(l) \neq \mathit{assign}(t)$; since lecture and tutorial slots are never equal, we read it as "must not overlap".

Special bookings (C_{13}): CPSC 851 and CPSC 913 never appear in the input; we create them. If the input has a CPSC 351 lecture, we add the tutorial t_{851} = `CPSC 851 TUT 01` to *Tutorials*, set $\mathit{partassign}(t_{851})$ = TT 18:00, and add $\mathit{notcompatible}(t_{851}, y)$ for every CPSC 351 lecture and tutorial y. CPSC 913 is created the same way, as t_{913}, when the input has a CPSC 413 lecture. C_{13} is then checked by C_7 and C_8. Like any other tutorial, these bookings count toward $\mathit{tutorialmax}$(TT 18:00), so if the input does not offer TT 18:00 with enough room for them, no valid timetable exists.

Validity: A complete timetable *assign* is valid if it breaks no hard constraint:

$$\boxed{\text{Constr}(\mathit{assign}) = \bigwedge_{i=1}^{13} C_i(\mathit{assign}).} \quad (18)$$

A partial assignment is *consistent* if the components placed so far break no constraint:

$$\text{Consistent}(\mathit{assign}) \iff \text{every } C_i \text{ holds on } \text{dom}(\mathit{assign}). \quad (19)$$

Once a partial assignment breaks a rule, placing more components cannot fix it: a full slot stays full and a clash stays a clash. So every model can discard a partial assignment as soon as Consistent fails.

2.3 Soft constraints and objective

Soft constraints may be broken, but each violation adds penalty points. They use these inputs:

- minimums $\mathit{lecturemin}(s)$ for $s \in \mathit{Slots}_L$ and $\mathit{tutorialmin}(s)$ for $s \in \mathit{Slots}_T$;

- $preference : (Lectures + Tutorials) \times Slots \to \mathbb{N}$ (0 where none is given);
- the $pair(x, y)$ statements, with $x, y \in X$;
- penalties $pen_{lecturemin}$, $pen_{tutorialmin}$, $pen_{notpaired}$, $pen_{section} \in \mathbb{N}$.

There are five penalties:

Too few lectures: If a lecture slot holds fewer lectures than its minimum, each missing lecture costs $pen_{lecturemin}$. A slot with minimum 3 that holds 1 lecture costs $2 \cdot pen_{lecturemin}$; a slot at or above its minimum costs nothing.

$$E_{L\,\min}(assign) = pen_{lecturemin} \cdot \sum_{s \in Slots_L} \max(0,\ lecturemin(s) - n_L(s)) \tag{20}$$

Too few tutorials: The same for tutorial slots.

$$E_{T\,\min}(assign) = pen_{tutorialmin} \cdot \sum_{s \in Slots_T} \max(0,\ tutorialmin(s) - n_T(s)) \tag{21}$$

Unmet preferences: If a component does not get a slot it has a preference for, that preference's value is added.

$$E_{\text{pref}}(assign) = \sum_{x \in X} \sum_{s \in Slots,\ s \neq assign(x)} preference(x, s) \tag{22}$$

Unmet pairs: If the two components of a $pair(x, y)$ statement are not in the same slot, this costs $pen_{notpaired}$. The instructor confirmed that inputs only pair two lectures or two tutorials, so "same time" in [2] means the same slot. The order does not matter: $pair(x, y)$ and $pair(y, x)$ count as one preference.

$$E_{\text{pair}}(assign) = pen_{notpaired} \cdot \left|\{\, \{x, y\} : pair(x, y),\ assign(x) \neq assign(y) \,\}\right| \tag{23}$$

Sections sharing a slot: Each pair of lecture sections of the same course placed in the same slot costs $pen_{section}$.

$$\begin{aligned} E_{\text{section}}(assign) = pen_{section} \cdot \big|\{\, \{l, l'\} \subseteq Lectures : l \neq l',\ course(l) = course(l'), \\ assign(l) = assign(l') \,\}\big| \end{aligned} \tag{24}$$

Objective: Eval is the total penalty,

$$\boxed{\begin{aligned} \text{Eval}(assign) = E_{L\,\min}(assign) + E_{T\,\min}(assign) + E_{\text{pref}}(assign) \\ + E_{\text{pair}}(assign) + E_{\text{section}}(assign), \end{aligned}} \tag{25}$$

and the problem is

$$\boxed{\min_{assign} \text{Eval}(assign) \quad \text{subject to} \quad \text{Constr}(assign) = \mathsf{true}.} \tag{26}$$

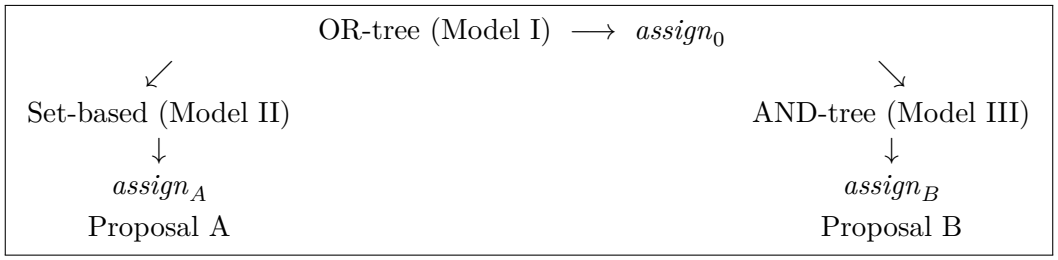
Every penalty is 0 or more, so a valid timetable with $\text{Eval}(assign) = 0$ is optimal.

3 Overall Search Architecture

3.1 How the models relate

We propose two complete search systems that share the same first stage:

- **Proposal A:** OR-tree for an initial valid timetable (Section 4) → set-based improvement (Section 5).
- **Proposal B:** OR-tree for an initial valid timetable (Section 4) → AND-tree optimization (Section 6).



(27)

All three models use the problem definition of Section 2: the components X, the domains $D(x)$, the overlap test (3), the checker Consistent (19) and Eval.

Shared first stage: Both second stages need a valid timetable to start from. The OR-tree (Section 4) finds one, $assign_0$, or reports that no valid timetable exists. It only looks for feasibility and ignores Eval.

How the stages connect: Both second stages begin with $assign_0$ as their best timetable and only replace it with a strictly better valid one, so

$$\text{Eval}(assign_A) \le \text{Eval}(assign_0) \quad \text{and} \quad \text{Eval}(assign_B) \le \text{Eval}(assign_0).$$

Why these models: We separate finding a valid timetable from improving it because the hard constraints are strict and many, while Eval only matters among timetables that are already valid. Finding a valid timetable is a constraint satisfaction problem, which an OR-tree handles directly: it places one component at a time, discards a branch as soon as Consistent fails, and either finds a valid timetable or shows that none exists. For the second stage we offer two options with different trade-offs. Set-based search (Proposal A) works only on complete valid timetables and changes them in small steps, so it always has a valid timetable to return, but it can stop at a local optimum. The AND-tree (Proposal B) searches all scheduling choices with branch-and-bound and can show that its result is optimal if it finishes, but on large inputs it may not finish in reasonable time. Both proposals use the same OR-tree first stage, so that part is built the same way whichever proposal is chosen.

4 Model I: OR-Tree Feasibility Search

4.1 Search model: $A_\vee = (S_\vee, T_\vee)$

Problem descriptions.

$$\boxed{Prob_\vee = \{(a, U) : a \text{ is partial},\ U = X \setminus \text{dom}(a)\}.} \tag{28}$$

Here a is the current partial timetable, and U contains unscheduled components. A node has the lecture's form

$$((a, U), sol), \qquad sol \in \{\mathsf{yes}, ?, \mathsf{no}\}. \tag{29}$$

Alternatives. Select a component $x \in U$ and define

$$D_a(x) = \{s \in D(x) : \text{Consistent}(a \cup \{x \mapsto s\})\}. \tag{30}$$

For every $s_i \in D_a(x)$, let

$$pr_i = (a \cup \{x \mapsto s_i\}, U \setminus \{x\}). \tag{31}$$

Then the application-specific alternative relation is

$$\boxed{Altern(pr, pr_1, \ldots, pr_r), \qquad r = |D_a(x)|.} \tag{32}$$

States and transitions. Following Lecture 11,

$$S_\vee \subseteq Otree, \qquad T_\vee = \{(q, q') : Erw_\vee(q, q') \text{ or } Erw^*_\vee(q, q')\}. \tag{33}$$

The extension relation marks a solved leaf yes, marks an unsolvable leaf no, or expands a leaf using *Altern*. Extensions can act on leaves inside a larger tree.

[**TO FILL:** Write the four formal $Erw_\vee$ cases from Lecture 11, using the definitions above.]

4.2 Search process: $P_\vee = (A_\vee, Env_\vee, K_\vee)$

[**TO FILL:** Define $Env_\vee$, leaf selection, and slot ordering. Proposed $K_\vee$: depth-first unfinished-leaf selection and MRV component selection. No forward checking is necessary for correctness.]

The MRV component selection is

$$\boxed{x^* = \arg\min_{x \in U} |D_a(x)|.} \tag{34}$$

4.3 Search instance: $Inst_\vee = (q_0, G_\vee)$

Start with

$$a_0 = \varnothing, \quad U_0 = X, \quad q_0 = ((a_0, U_0), ?). \tag{35}$$

A leaf is solved if $U = \varnothing$ and Const(a) is true. The search halts when a solved branch exists or all alternatives have failed; these are distinct outcomes.

[**TO FILL:** Define $G_\vee$ recursively in terms of the tree and distinguish success from exhaustive failure.]

4.4 Algorithm 1: OR-tree feasibility

Algorithm 1 OR-Tree Feasibility Search (operational outline)

Require: Components X, domains D, partial checker Consistent
Ensure: Complete valid timetable, or FAILURE

Initialize tree $q \leftarrow ((\varnothing, X), ?)$
while an unresolved leaf exists **do**
    $v \leftarrow$ deepest unresolved leaf of q
    $(a, U) \leftarrow$ problem description at v
    if $U = \varnothing$ **then**
        if Const(a) **then**
            Mark v as yes; **return** a
        else
            Mark v as no;
            continue
        end if
    end if
    $x \leftarrow \arg\min_{y \in U} |D_a(y)|$ ▷ MRV
    $C \leftarrow D_a(x)$
    if $C = \varnothing$ **then**
        Mark v as no
    else
        for all $s \in C$ in deterministic slot order **do**
            Add child $((a \cup \{x \mapsto s\}, U \setminus \{x\}), ?)$ to v
        end for
    end if
end while
return FAILURE

4.5 Worked OR-tree derivation

[**TO FILL:** Starting with $a_0 = \varnothing$ and $U_0 = X$, show every intermediate (a_i, U_i); draw an OR-tree with at least one failing alternative and one solved leaf. Explain why the search stops on its first successful path.]

5 Model II: Set-Based Local Optimization

Now, we need a second model that will use the valid timetable produced by the OR-tree, and produce an output minimizing the soft penalties applied due to the soft constraints we have in the problem.

Our best case scenario would be choosing a solution that has a zero penalty, otherwise we would like the schedule that has the least penalties.

Set-based search works in this case because it allows for small improvements on the solution pass by pass.

5.1 Search model: $A_{set} = (S_{set}, T_{set})$

Our search model is:

$$A_{set} = (S_{set}, T_{set}) \tag{36}$$

where:

- A_{set} is our set-based search model.
- S_{set} is the state space for the search.
- $T_{set} \subseteq S_{set} \times S_{set}$ defines the transitions between states.

In order to define the states and transitions, we first need to establish the facts that make up our timetables, along with the rules that allow us to modify them.

Facts. Following Lecture 06, we define our set of facts as:

$$F = \{\text{Assigned}(x, s) \mid x \in X,\ s \in D(x)\} \tag{37}$$

where:

- F is the universe of all possible assignment facts.
- $\text{Assigned}(x, s)$ means that component x is assigned to time slot s.
- X is the set of all components, including lectures and tutorials.
- $D(x)$ is the set of allowed time slots for component x.

Each fact is simply one assignment in our timetable. For example, $\text{Assigned}(L_1, s_9)$ would mean that lecture L_1 has been scheduled into slot s_9.

By combining these facts, we can represent an entire timetable as a single set.

States. Lecture 06 defines the state space as a subset of the power set of facts:

$$S_{set} \subseteq 2^F \tag{38}$$

For our problem, we only want to consider states that represent complete, valid timetables. Therefore, we define:

$$S_{set} = \left\{ q \subseteq F \;\middle|\; \begin{array}{l} q \text{ assigns every } x \in X \text{ exactly once,} \\ \text{Const}(a_q) = \text{true} \end{array} \right\} \tag{39}$$

Here, q represents a set of assignment facts, while a_q is the actual timetable represented by those facts.

In other words, every state in our search is already a valid solution to the constraint satisfaction problem. We are not trying to schedule more components at this stage, but rather trying to improve the assignments that have already been made.

This also means that we do not need to generate every possible valid timetable beforehand. We only maintain one current timetable and generate new states as the search progresses.

Extensions. Now that we have our states, we need a way to move between them.

Lecture 06 defines extension rules as:

$$EXT \subseteq \{A \longrightarrow B \mid A, B \subseteq F\} \tag{40}$$

Here, A is a set of facts that we want to remove, and B contains the facts that will replace them.

For our model, we propose two types of extensions: Move and Swap.

1. Move

A Move extension changes the time slot assigned to a single component.

$$\{\text{Assigned}(x, s_1)\} \longrightarrow \{\text{Assigned}(x, s_2)\} \tag{41}$$

Here, x is the component being moved, s_1 is its current time slot, and s_2 is the new time slot.

For example, we could move CPSC 231 LEC 02 from MWF 9–10 to MWF 10–11, provided that doing so does not violate any hard constraints.

2. Swap

A Swap extension exchanges the assigned slots of two components.

$$\begin{aligned} &\{\text{Assigned}(x, s_1), \text{Assigned}(y, s_2)\} \\ &\quad \longrightarrow \{\text{Assigned}(x, s_2), \text{Assigned}(y, s_1)\} \end{aligned} \tag{42}$$

Here, x and y are two distinct components, while s_1 and s_2 are their current slots.

We use swaps because sometimes moving one component on its own may not be possible, but exchanging the slots of two components could still produce a better timetable.

For both extensions, we need to ensure that the new assignments belong to the components' allowed domains, respect preassignments, and do not violate any hard constraints.

This is important because we do not want our optimization process to undo the work already done by the OR-tree.

Transitions. Once we select an extension rule, we can apply it to the current state using the transition defined in Lecture 06:

$$q' = (q \setminus A) \cup B \tag{43}$$

where:

- q is our current state.
- A contains the facts we are removing.
- B contains the replacement facts.
- q' is the resulting state.

Our transition relation is therefore:

$$\begin{aligned} T_{set} = \{(q, q') \mid \exists (A \to B) \in EXT, \\ A \subseteq q, \\ q' = (q \setminus A) \cup B, \\ q' \in S_{set}\} \end{aligned} \tag{44}$$

The last condition, $q' \in S_{set}$, is particularly important. It ensures that every transition leads to another valid timetable.

Therefore, our set-based search only moves between solutions that satisfy the hard constraints.

5.2 Search process: $P_{set} = (A_{set}, Env_{set}, K_{set})$

Now that we have defined which transitions are possible, we need to decide which transitions should actually be performed.

Following Lecture 06, our search process is:

$$P_{set} = (A_{set}, Env_{set}, K_{set}) \tag{45}$$

where:

- A_{set} is our search model.
- Env_{set} contains the information required by the search.
- K_{set} is the search control that decides which extension to apply.

Environment. Our environment contains the information needed to check and evaluate our timetables. This includes the allowed slot domains, hard constraints, preferences, and the penalty values for each soft constraint.

The current state q is maintained by the search process and is available when evaluating possible extensions.

Search control. For our control, we propose a greedy hill-climbing approach.

The idea is fairly simple. At every step, we consider the available Move and Swap extensions, calculate the resulting penalties, and choose the extension that gives us the lowest evaluation score.

However, we only apply an extension if it actually improves the current timetable.

Weight function

Lecture 06 uses f_{weight} to assign a numerical value to each applicable extension rule.

For our problem, we define:

$$f_{weight}(A, B, e) = \text{Eval}\left(a_{(q \setminus A) \cup B}\right) \tag{46}$$

Here, e provides the evaluation information needed to calculate the result of the proposed extension, while q is the current search state.

The function essentially asks: if we perform this Move or Swap, what will the new timetable's penalty be?

Since our objective is to minimize penalties, a lower weight is preferable.

Applicable extensions

We define the set of currently applicable extensions as:

$$\begin{aligned} R(q) = \{(A, B) \mid\ & A \to B \in EXT, \\ & A \subseteq q, \\ & (q \setminus A) \cup B \in S_{set}, \\ & (q \setminus A) \cup B \neq q\} \end{aligned} \tag{47}$$

Here, $R(q)$ contains all Move and Swap extensions that can be applied to our current state while still producing a different valid timetable.

If $R(q)$ is nonempty, we find the minimum weight:

$$w_{\min}(q) = \min_{(A,B) \in R(q)} f_{weight}(A, B, e) \tag{48}$$

This gives us the lowest penalty achievable through one extension from the current state.

Selection function

It is possible that multiple extensions produce the same minimum weight.

In that case, we use the function f_{select} from Lecture 06 to break the tie.

First, we collect the best extensions:

$$R_{\min}(q) = \{(A, B) \in R(q) \mid f_{weight}(A, B, e) = w_{\min}(q)\} \tag{49}$$

We then select one:

$$(A^*, B^*) = f_{select}(R_{\min}(q), e) \tag{50}$$

Our proposed tie-breaker simply chooses randomly among the equally weighted extensions.

This gives us one extension to consider applying.

Strict improvement

There is one additional condition we need.

Suppose our current timetable has a penalty of 10, but the best available extension would increase it to 12. We obviously do not want to make that change.

Therefore, we only perform the selected extension when:

$$w_{\min}(q) < \text{Eval}(a_q) \tag{51}$$

If this condition holds, the next state becomes:

$$K_{set}(q, e) = (q \setminus A^*) \cup B^* \tag{52}$$

Otherwise, we stop the search and return the current timetable.

Neighborhood. We can also describe the possible next states using the idea of a neighborhood.

For a state q, define:

$$N(q) = \{q' \in S_{set} \mid (q, q') \in T_{set},\ q' \neq q\} \tag{53}$$

Here, $N(q)$ contains every valid timetable that can be reached from q through one Move or Swap.

Our hill-climbing control looks through this neighborhood and chooses the timetable with the lowest evaluation score, as long as it improves on the current one.

Formally, an improvement exists when:

$$\exists q' \in N(q) : \text{Eval}(a_{q'}) < \text{Eval}(a_q) \tag{54}$$

Local optimum. One problem with this approach is that we might eventually reach a timetable where no single Move or Swap can improve the penalty.

This is called a local optimum.

Formally:

$$\forall q' \in N(q), \quad \text{Eval}(a_{q'}) \geq \text{Eval}(a_q) \tag{55}$$

However, this does not necessarily mean we have found the best timetable possible.

There could still be a better solution somewhere else in the search space, but our current extension rules might not allow us to reach it through a single improvement.

This is the main limitation of using a greedy approach for optimization.

5.3 Search instance: $Inst_{set} = (q_0, G_{set})$

Following Lecture 06, a search instance is defined as:

$$Inst_{set} = (q_0, G_{set}) \tag{56}$$

where q_0 is the initial state and G_{set} is our goal condition.

Initial state. Since the OR-tree has already produced a valid timetable, we do not need to start with an empty state.

Let a_{feasible} represent the timetable returned by the OR-tree.

We convert it into a set of assignment facts:

$$q_0 = \{\text{Assigned}(x, a_{\text{feasible}}(x)) \mid x \in X\} \tag{57}$$

This means that every component in the timetable becomes one fact in our initial set-based state.

Since the OR-tree has already satisfied all hard constraints, we know that:

$$\text{Const}(a_{\text{feasible}}) = \text{true} \tag{58}$$

Therefore, $q_0 \in S_{set}$, and we can begin optimizing immediately.

Goal condition. Our ideal situation is finding a timetable with an evaluation score of zero.

Since penalties cannot be negative, a score of zero would be globally optimal.

However, we also need to account for the possibility that no further improvement is available.

Our goal condition is therefore:

$$\begin{aligned} G_{set}(q) = \textsf{yes} \iff\ & \text{Eval}(a_q) = 0 \\ & \vee \neg\exists q' \in N(q) : \text{Eval}(a_{q'}) < \text{Eval}(a_q) \end{aligned} \tag{59}$$

In other words, we stop when we either find a zero-penalty timetable or reach a local optimum.

For larger instances, we could also introduce a time or iteration limit to ensure that the search finishes within a reasonable amount of time.

5.4 Algorithm 2: Set-based hill climbing

The following algorithm summarizes how our search process would work.

We start with the OR-tree's valid timetable, generate all possible valid neighbors through Move and Swap extensions, and select the best improvement.

Algorithm 2 Set-Based Optimization

Require: Feasible timetable a_{feasible}, EXT, Eval
Ensure: Improved valid timetable

$q \leftarrow \{\text{Assigned}(x, a_{\text{feasible}}(x)) \mid x \in X\}$
while $\text{Eval}(a_q) > 0$ **do**
  $R \leftarrow R(q)$
  if $R = \varnothing$ **then**
    return a_q
  end if
  $m \leftarrow \min_{(A,B)\in R} f_{weight}(A, B, e)$
  if $m \geq \text{Eval}(a_q)$ **then**
    return a_q ▷ Local optimum
  end if
  $R_{\min} \leftarrow \{(A, B) \in R \mid f_{weight}(A, B, e) = m\}$
  $(A^*, B^*) \leftarrow f_{select}(R_{\min}, e)$
  $q \leftarrow (q \setminus A^*) \cup B^*$
end while
return a_q

The algorithm keeps improving the timetable for as long as it finds a valid extension that reduces the evaluation score.

At every iteration, we generate the applicable extensions, calculate their penalties, and use our selection function to choose the best one.

If no extension improves the score, we stop and return the current timetable.

Since our states and extension rules are restricted to valid timetables, we also ensure that no hard constraints are violated during the optimization process.

5.5 Worked set-based derivation

To demonstrate how our search works, consider a simplified scheduling problem involving two lecture sections and two tutorials for CPSC 231.

We define:

$$L_1 = \text{CPSC 231 LEC 01} \tag{60}$$
$$L_2 = \text{CPSC 231 LEC 02} \tag{61}$$
$$T_1 = \text{CPSC 231 TUT 01} \tag{62}$$
$$T_2 = \text{CPSC 231 TUT 02} \tag{63}$$

We also define the relevant weekly time slots:

$$s_9 = \text{MWF } 9\text{–}10 \tag{64}$$
$$s_{10} = \text{MWF } 10\text{–}11 \tag{65}$$
$$s_{11} = \text{MWF } 11\text{–}12 \tag{66}$$
$$r_{10} = \text{TR } 10\text{–}11 \tag{67}$$
$$w_{11} = \text{MW } 11\text{–}12 \tag{68}$$

For this example, suppose the allowed domains are:

$$D(L_1) = \{s_9\} \tag{69}$$
$$D(L_2) = \{s_9, s_{10}, s_{11}\} \tag{70}$$
$$D(T_1) = \{r_{10}\} \tag{71}$$
$$D(T_2) = \{w_{11}\} \tag{72}$$

We will assume that the lecture room capacity is two, the tutorial room capacity is one, and both tutorials are shared by both lecture sections.

Therefore, tutorials cannot overlap either lecture.

For our soft constraints, suppose we apply a penalty of 5 when both lecture sections occupy the same time slot. All other soft penalties are assumed to be zero for this example.

Our evaluation function becomes:

$$\text{Eval}(a) = 5 \cdot \mathbf{1}[a(L_1) = a(L_2)] \tag{73}$$

where $\mathbf{1}[P]$ equals 1 when P is true, and 0 otherwise.

Initial timetable. Suppose our OR-tree search returns the following timetable:

Component	Assigned Slot
CPSC 231 LEC 01	MWF 9–10
CPSC 231 LEC 02	MWF 9–10
CPSC 231 TUT 01	TR 10–11
CPSC 231 TUT 02	MW 11–12

This timetable is valid because both lectures can share the slot under our assumed room capacity, and neither tutorial overlaps the lectures.

However, both lecture sections are scheduled at the same time, giving us a penalty of 5.

$$\text{Eval}(a_{\text{feasible}}) = 5 \tag{74}$$

Converting the timetable into facts. Our initial set-based state is:

$$\begin{aligned} q_0 = \{ & \text{Assigned}(L_1, s_9), \\ & \text{Assigned}(L_2, s_9), \\ & \text{Assigned}(T_1, r_{10}), \\ & \text{Assigned}(T_2, w_{11})\} \end{aligned} \tag{75}$$

Since this is a valid timetable:

$$q_0 \in S_{set} \tag{76}$$

And its evaluation score is:

$$\text{Eval}(a_{q_0}) = 5 \tag{77}$$

Finding an improvement. Our search control now considers the possible Move and Swap extensions.

One possible move is shifting L_2 from MWF 9–10 to MWF 10–11.

For this extension:

$$A = \{\text{Assigned}(L_2, s_9)\} \tag{78}$$

$$B = \{\text{Assigned}(L_2, s_{10})\} \tag{79}$$

Therefore:

$$\{\text{Assigned}(L_2, s_9)\} \longrightarrow \{\text{Assigned}(L_2, s_{10})\} \tag{80}$$

This extension is valid because the new slot belongs to $D(L_2)$ and does not overlap either tutorial.

On the other hand, moving L_2 to MWF 11–12 would not be permitted, since that slot overlaps T_2 on Monday and Wednesday.

Applying the extension. Using the set-based transition rule:

$$q_1 = (q_0 \setminus A) \cup B \tag{81}$$

We obtain:

$$\begin{aligned} q_1 = \{ & \text{Assigned}(L_1, s_9), \\ & \text{Assigned}(L_2, s_{10}), \\ & \text{Assigned}(T_1, r_{10}), \\ & \text{Assigned}(T_2, w_{11})\} \end{aligned} \tag{82}$$

Our new timetable is:

Component	Assigned Slot
CPSC 231 LEC 01	MWF 9–10
CPSC 231 LEC 02	MWF 10–11
CPSC 231 TUT 01	TR 10–11
CPSC 231 TUT 02	MW 11–12

This timetable still satisfies every hard constraint:

$$\text{Const}(a_{q_1}) = \text{true} \tag{83}$$

However, the two lecture sections now occupy different time slots, meaning the soft penalty is removed.

$$\text{Eval}(a_{q_1}) = 0 \tag{84}$$

Comparing the two states:

$$\text{Eval}(a_{q_1}) < \text{Eval}(a_{q_0}) \tag{85}$$

$$0 < 5 \tag{86}$$

Therefore, our hill-climbing control accepts this extension.

Termination. Since we have reached an evaluation score of zero, our goal condition is satisfied:

$$G_{set}(q_1) = \textsf{yes} \tag{87}$$

The search terminates and returns the improved timetable.

The complete derivation is:

$$q_0 \xrightarrow{\text{Move } L_2 : s_9 \to s_{10}} q_1 \tag{88}$$

With the evaluation score changing from:

$$5 \longrightarrow 0 \tag{89}$$

This example shows how the set-based search takes a valid timetable from the OR-tree, applies a small change, and produces a better solution without violating any hard constraints.

5.6 Justification and limitations

We chose set-based search because it fits naturally with the second stage of our scheduling problem.

By the time this search begins, all components have already been assigned and the hard constraints are satisfied. Rather than rebuilding the timetable, we can focus on improving the assignments through smaller changes.

Our Move and Swap extensions also give us a straightforward way to explore different schedules, while the weight function allows us to compare them using the evaluation function defined in the problem.

However, the main limitation comes from our greedy search control.

Hill climbing only accepts extensions that improve the current timetable. This means we may eventually reach a local optimum, even when a better timetable exists elsewhere.

There is also the possibility that certain improvements require multiple components to move together. Since our current extensions only move one component or swap two, the search may not always be able to reach those solutions.

We could address this later through more advanced controls, such as simulated annealing, or by introducing extension rules that modify several assignments at once.

For our proposed model, however, greedy hill climbing gives us a relatively simple optimization process that follows the set-based search framework from Lecture 06 while keeping every intermediate timetable valid.

6 Solution 2: AND-Tree Optimization Following OR-Tree Feasibility

6.1 Purpose and connection to the feasibility stage

The second stage receives a complete feasible timetable from OR-tree search and seeks a timetable with minimum soft-constraint penalty. We use the optimization interpretation of AND-tree search from Lecture 09: divide an optimization problem into cases, account for every case, and combine their results by comparison. Branch-and-bound avoids exploring a case when a sound lower bound proves that it cannot improve the best timetable already found.

The children of a division represent different possible assignments of one offering. Their timetables are compared, rather than combined into a single assignment. All child cases must nevertheless be accounted for before the parent's optimization task is resolved. This differs from the preceding OR-tree feasibility stage, which may stop after finding one valid completion.

Let a_{init} be the timetable returned by the feasibility stage. The optimization stage initializes its incumbent and upper bound as

$$a_{\text{best}} = a_{\text{init}}, \qquad B = \text{Eval}(a_{\text{best}}). \tag{90}$$

The incumbent is always complete and satisfies every hard constraint. Its assignments are not imposed on the optimization tree. Only assignments fixed by the input or required by the problem specification are fixed at the root. If the feasibility stage proves that no valid timetable exists, this stage is not invoked. If that stage stops without finding a timetable or proving infeasibility, its result is inconclusive and does not establish that the input is infeasible.

6.2 Scheduling notation and shared functions

Let L be the set of lecture sections, T the set of tutorials and labs, and $X = L \cup T$. Let S be the set of typed abstract weekly slots and $D(x) \subseteq S$ the domain of offering x. A slot records its offering type, recurring days, and time interval. Actual time overlap is used for hard conflict checks, including when lecture and tutorial durations differ.

An assignment a is a partial function from X to S, with $a(x) \in D(x)$ for each $x \in \mathrm{dom}(a)$. Define

$$U(a) = X \setminus \mathrm{dom}(a). \tag{91}$$

A complete assignment $\widehat{a}$ extends a, written $\widehat{a} \supseteq a$, when it agrees with every assignment already recorded in a.

We use $\mathrm{Constr}(\widehat{a})$ for the conjunction of all hard constraints in the official problem statement. We use $\mathrm{Consistent}(a)$ for a sound partial-assignment check: it rejects known violations without rejecting a partial assignment that has a feasible completion. Passing this check does not prove that a feasible completion exists. For complete assignments, the check agrees with Constr. Both stages use the same domains, constraint interpretations, and full evaluation function.

Let a_{fix} contain all mandatory assignments. Preprocessing creates the CPSC 851 tutorial when a CPSC 351 lecture is present and the CPSC 913 tutorial when a CPSC 413 lecture is present. Each required special tutorial is created once, is fixed to Tuesday/Thursday 18:00–19:00, and counts toward the ordinary tutorial capacity. The prescribed conflicts with its associated course are included in Constr. These generated offerings are included in T before either search begins. All partial assignments considered below extend a_{fix}.

For $x \in U(a)$, define its currently legal slots by

$$D_a(x) = \{s \in D(x) : \mathrm{Consistent}(a \cup \{x \mapsto s\})\}. \tag{92}$$

The full objective $\mathrm{Eval}(\widehat{a})$ is the sum of the specified minimum-lecture-usage, minimum-tutorial-usage, preference, pairing, and lecture-section penalties. Every term is nonnegative. A low objective value never compensates for a hard-constraint violation.

6.3 Search model

The model is $A_\wedge = (S_\wedge, T_\wedge)$.

Problem descriptions. Define

$$\mathrm{Prob}_\wedge = \{\mathrm{Opt}(a, U) : a \text{ is a partial assignment extending } a_{\mathrm{fix}}, U = U(a)\}. \tag{93}$$

The description $\mathrm{Opt}(a, U)$ asks us to account for all valid completions of a when searching for a minimum-penalty timetable. Its underlying optimization value is

$$\mathcal{F}(a) = \{\widehat{a} : X \to S : \widehat{a} \supseteq a, \mathrm{Constr}(\widehat{a}) = \mathsf{true}\}, \tag{94}$$

$$V(a, U) = \min_{\widehat{a} \in \mathcal{F}(a)} \mathrm{Eval}(\widehat{a}), \qquad \min \varnothing = +\infty. \tag{95}$$

The value $+\infty$ denotes absence of a feasible completion. It is not an additional soft penalty. The bounded search need not compute the exact value of every subproblem: a certified lower bound can suffice to exclude a case from improving the global incumbent.

Division relation. For an unresolved problem $p = \mathrm{Opt}(a, U)$, choose $x \in U$. If $D_a(x) = \{s_1, \ldots, s_k\}$ with $k \geq 1$, define

$$p_i = \mathrm{Opt}(a \cup \{x \mapsto s_i\}, U \setminus \{x\}), \qquad 1 \leq i \leq k, \tag{96}$$

$$\mathrm{Div}(p, p_1, \ldots, p_k) \quad \text{holds.} \tag{97}$$

Each choice of the offering x gives a possible division. Once selected, that division contains every currently legal slot case for x. It does not select just one slot. These cases partition the feasible completions of the parent, so

$$V(a, U) = \min_{1 \le i \le k} V(a \cup \{x \mapsto s_i\}, U \setminus \{x\}). \tag{98}$$

The full partial assignment is retained in every child; shared capacities and conflicts between offerings are therefore preserved. No independence between departments or courses is assumed. If any unassigned offering has an empty legal domain, then $\mathcal{F}(a) = \varnothing$ and the case can be resolved without a division.

Tree states and resolution certificates. As in the lecture, states are whole AND-trees, $S_\wedge \subseteq \mathit{Atree}$. A leaf is (p, sol) and an internal node is $(p, \mathit{sol}, b_1, \ldots, b_k)$, where $p \in \mathrm{Prob}_\wedge$ and $\mathit{sol} \in \{?, \mathsf{yes}\}$. The environment stores the incumbent and the certificates associated with tree nodes.

For this optimization instantiation, yes means that the node's case is resolved for the global minimization task. It does not necessarily mean that the case contains a feasible timetable or that its exact optimum was computed. A resolved node v with problem $\mathrm{Opt}(a, U)$ has a retained certificate ℓ_v satisfying

$$B \le \ell_v \le V(a, U). \tag{99}$$

Thus no completion in that case improves the current incumbent. Certificates are obtained in the following ways:

1. For a complete valid assignment, evaluate it and update the incumbent if its penalty is smaller. Then take $\ell_v = \mathrm{Eval}(a)$.
2. For a case proved infeasible, take $\ell_v = +\infty$.
3. For a case pruned by a sound lower bound, take $\ell_v = \mathrm{LB}(a)$.
4. If every child of a division is resolved, take $\ell_v = \min_i \ell_{v_i}$. Equation (98) makes this a sound certificate for the parent.

The incumbent value only decreases, so certificates remain valid after a later improvement. A pruned case is recorded as dominated, not incorrectly reported as infeasible or assigned an exact value of $+\infty$.

Expansion and contraction. We instantiate the lecture's extension operations as follows:

$$(p, ?) \longrightarrow (p, \mathsf{yes}) \qquad \text{when a certificate is available,} \tag{100}$$

$$(p, ?) \longrightarrow (p, ?, (p_1, ?), \ldots, (p_k, ?)) \qquad \text{when } \mathrm{Div}(p, p_1, \ldots, p_k). \tag{101}$$

These local operations, and their application within an unchanged surrounding tree, define $\mathrm{Erw}_\wedge$. A solved leaf is not expanded again. For this section, $\mathrm{Erw}_\wedge^*$ denotes a finite sequence of such extensions, including the zero-step sequence. Reversing an extension sequence gives a structural contraction:

$$T_\wedge = \{(q, q') \in S_\wedge \times S_\wedge : \mathrm{Erw}_\wedge(q, q') \text{ or } \mathrm{Erw}_\wedge^*(q', q)\}. \tag{102}$$

The argument order in the second relation expresses contraction. The control permits contraction of a resolved subtree only after retaining its aggregate certificate and any incumbent update. It may then mark the resulting leaf yes. It never erases an unresolved case or repeats a completed division. Ordinary depth-first return proceeds to the next unresolved sibling. Infeasible children resolve to $+\infty$; they do not force the entire parent optimization problem to fail.

6.4 Search process

The process is $P_\wedge = (A_\wedge, Env_\wedge, K_\wedge)$.

Environment. The environment contains the fixed scheduling instance, D, the shared constraint checker and objective, a_{best}, B, node certificates, and traversal records. Fixed total orders on offerings and slots give deterministic tie-breaking. The environment also records whether processing finished normally or was interrupted by a specified resource limit.

A sound lower bound. Let $w(x, s) \geq 0$ be the input preference value attached to offering x and preferred slot s, or zero if no such preference is supplied. Let $\mathcal{P}$ be the set of requested unordered offering pairs, with reversed duplicates counted once. By the instructor's clarification, each pair contains two lectures or two tutorials/labs. A pairing is satisfied exactly when their assigned slots are equal. Let $\mathcal{R}$ contain every unordered pair of distinct lecture sections with the same department and course number.

Write $\mathbf{1}[Q]$ for 1 when proposition Q is true and 0 otherwise, and put $A = \text{dom}(a)$. Define

$$\text{LB}(a) = \sum_{x \in A} \sum_{s \in S} w(x, s)\, \mathbf{1}[a(x) \neq s] \tag{103}$$

$$+ \text{pen}_{\text{notpaired}} \sum_{\substack{\{x,y\} \in \mathcal{P} \\ x,y \in A}} \mathbf{1}[a(x) \neq a(y)] \tag{104}$$

$$+ \text{pen}_{\text{section}} \sum_{\substack{\{x,y\} \in \mathcal{R} \\ x,y \in A}} \mathbf{1}[a(x) = a(y)]. \tag{105}$$

Here $\text{pen}_{\text{section}}$ denotes the specified penalty for two sections of the same course sharing a slot. All counted penalties are fixed by assignments already made and persist in every extension. Minimum-occupancy penalties are omitted because additional offerings may remove the current shortfall. They are still included when computing the full Eval of a complete timetable. Since the remaining objective contributions are nonnegative,

$$\text{LB}(a) \leq \text{Eval}(\widehat{a}) \quad \text{for every } \widehat{a} \in \mathcal{F}(a). \tag{106}$$

Consequently the pruning condition is

$$\text{LB}(a) \geq B. \tag{107}$$

Equality is sufficient because the task is to return one minimum-penalty timetable, rather than enumerate all timetables tied for the minimum.

Leaf selection: f_{leaf}. Select the next unresolved leaf in a depth-first traversal. When a division is first created, order its children by increasing $\text{LB}(a \cup \{x \mapsto s\})$, breaking ties by the fixed slot order. Fully process the first child before continuing to its next sibling. This seeks a useful complete incumbent early while avoiding a global best-first frontier. Resolved leaves are excluded from selection.

Transition selection: f_{tran}. For the selected leaf $\text{Opt}(a, U)$, apply the first applicable rule in the following order:

1. If $\text{Consistent}(a)$ is false, resolve the case as infeasible.

2. If $U = \varnothing$, check Constr(a). Evaluate and possibly replace the incumbent if valid; otherwise resolve as infeasible.

3. If some $x \in U$ has $D_a(x) = \varnothing$, resolve as infeasible.

4. If $\mathrm{LB}(a) \geq B$, retain the lower-bound certificate and resolve the case without expansion.

5. Otherwise select the division offering by minimum remaining values:

$$x^* = \arg\min_{x \in U} |D_a(x)|, \tag{108}$$

breaking ties by the fixed offering order. Expand into all legal slot cases for x^* using (97).

After a case is resolved, combine certificates upward whenever all children of a node have been resolved. No alternative division by another offering is required: the chosen division already covers every feasible completion. Together these rules define $K_\wedge$.

6.5 Search instance and termination

The search instance is $\mathrm{Inst}_\wedge = (q_0, G_\wedge)$, where

$$q_0 = (\mathrm{Opt}(a_{\mathrm{fix}}, X \setminus \mathrm{dom}(a_{\mathrm{fix}})), ?). \tag{109}$$

Initialize the environment by (90). Define tree resolution recursively by

$$\mathrm{Resolved}((p, \mathsf{yes})) = \mathsf{true}, \tag{110}$$

$$\mathrm{Resolved}((p, ?)) = \mathsf{false}, \tag{111}$$

$$\mathrm{Resolved}((p, ?, b_1, \ldots, b_k)) = \bigwedge_{i=1}^{k} \mathrm{Resolved}(b_i). \tag{112}$$

Every yes marking must be supported by a retained certificate. For reachable states, the goal predicate is

$$G_\wedge(q) = \mathrm{Resolved}(q). \tag{113}$$

The comparison operation in (98) provides the required compatibility of child results: they concern an exhaustive partition of the parent's completion set. Their distinct timetable assignments need not coexist in a single schedule.

When the root is resolved, return a_{best} as optimal. A zero incumbent permits immediate termination: all objective terms are nonnegative, so the root has the certificate $\ell_{\mathrm{root}} = 0$. If a resource limit interrupts the search before resolution, return the incumbent as feasible, with optimality not established. Such interruption is an external stopping condition, not satisfaction of $G_\wedge$.

6.6 Operational procedure

The following recursive procedure implements the selected divisions and depth-first control. The incumbent is shared across recursive calls. The return value is a certificate, not necessarily the exact optimum of that subproblem. An interruption propagates immediately to the driver; unfinished parents receive no certificate.

Algorithm 1 AND-Tree Branch-and-Bound: Recursive Search

Require: Partial assignment a, remaining offerings U; shared incumbent a_{best} and bound B.
Ensure: Return a certificate, or propagate interruption.

```
function OptimizeCase(a, U)
  if resource limit reached then interrupt
  if ¬ Consistent(a) then return ∞
  if U = ∅ then
    if ¬ Constr(a) then return ∞
    v ← Eval(a)
    if v < B then
      a_best ← copy(a)
      B ← v
    end if
    return v
  end if
  Compute D_a(x) for every x ∈ U
  if some D_a(x) = ∅ then return ∞
  b ← LB(a)
  if b ≥ B then return b
  x ← MRV offering, with fixed tie-breaking
  L ← D_a(x), ordered by child lower bound, then slot order
  c ← ∞
  for each s ∈ L do
    a' ← a ∪ {x ↦ s}
    c' ← OptimizeCase(a', U \ {x})
    c ← min(c, c')
  end for
  return c
end function
```

Algorithm 2 AND-Tree Optimization: Driver

Require: Instance I and complete timetable a_{init} with $\text{Constr}(a_{\text{init}}) = \mathsf{true}$.
Ensure: Return the best feasible timetable found and its optimality status.

```
function Driver(I, a_init)
  a_best ← copy(a_init)
  B ← Eval(a_best)
  if B = 0 then return (a_best, OPTIMAL)
  try
    OptimizeCase(a_fix, X \ dom(a_fix))
    return (a_best, OPTIMAL)
  on interruption
    return (a_best, FEASIBLE_OPTIMALITY_NOT_ESTABLISHED)
  end try
end function
```

6.7 Correctness and limitations

Feasibility and preservation of the incumbent. The initial incumbent is feasible by the first stage's contract. It is replaced only by a complete assignment passing Constr. Therefore every returned timetable is feasible and its penalty is no larger than that of a_{init}.

Coverage, pruning, and optimality. For any selected offering, every feasible completion belongs to exactly one child slot case. Sound partial consistency checking never removes a feasible completion. Equation (106) justifies every bound-based pruning decision. Terminal and combined certificates satisfy (99); decreases in B preserve previously obtained certificates. When the root is resolved,

$$B \leq \ell_{\text{root}} \leq V(a_{\text{fix}}, U(a_{\text{fix}})) \leq B. \tag{114}$$

The final inequality holds because the incumbent is itself a feasible completion of a_{fix}. Hence B equals the global minimum.

Termination and cost. Every division assigns one previously unassigned offering, so branch depth is at most $|X \setminus \text{dom}(a_{\text{fix}})|$. Domains are finite, and the control never revisits a resolved case. Without interruption, the search therefore terminates. Branch-and-bound can reduce exploration, but worst-case work remains exponential in the number of offerings. MRV and child ordering change exploration order without sacrificing coverage. The initial feasible timetable supplies both a usable result on interruption and an upper bound from the beginning; a better initial bound may permit more pruning. A depth-first implementation can release resolved subtree detail after preserving its certificate, rather than retain the entire explored tree. No claim of consistently fast execution or optimality under a time limit is made.

6.8 Demonstration on a small search instance

This instance demonstrates the AND-tree stage of Solution 2, using a feasible timetable supplied by the preceding OR-tree stage. It illustrates division, leaf and transition selection, incumbent improvement, pruning, and resolution of the root.

Instance and initial incumbent. Let $X = L = \{x, y, z\}$ and $T = \varnothing$, where x, y, z are lecture sections of three different courses. There are two lecture slots, s_1 = MWF 09:00–10:00 and s_2 = MWF 10:00–11:00, with $D(x) = D(y) = D(z) = \{s_1, s_2\}$. Each slot has lecture maximum 2 and lecture minimum 0. The only additional active hard restriction is noncompatible(x, y), so $a(x) \neq a(y)$. There are no mandatory assignments, unwanted slots, active-learning requirements, pairing requests, same-course section pairs, or special course bookings; all other hard restrictions are inactive for this illustrative instance. Thus $a_{\text{fix}} = \varnothing$.

The preference data are

$$w(x, s_1) = 4, \qquad w(y, s_1) = 2, \qquad w(z, s_1) = 1,$$

with all other preference values zero. Because every other soft term vanishes here, a complete feasible timetable has score

$$\text{Eval}(a) = 4\mathbf{1}[a(x) = s_2] + 2\mathbf{1}[a(y) = s_2] + \mathbf{1}[a(z) = s_2].$$

The lower bound counts exactly those preference penalties already fixed by a partial assignment. Use offering order $x < y < z$ and slot order $s_1 < s_2$ for ties.

Suppose the OR-tree stage returns

$$a_{\text{init}} = \{x \mapsto s_2, y \mapsto s_1, z \mapsto s_2\}, \qquad B = \text{Eval}(a_{\text{init}}) = 5.$$

This timetable satisfies the hard restrictions: x and y use different slots, and the largest occupancy is 2. Its choices supply an upper bound; they do not fix the optimization root.

First division and depth-first selection. The initial state is $q_0 = (\text{Opt}(\varnothing, \{x, y, z\}), ?)$. Its lower bound is 0 and all three legal domains have size 2. Consequently f_{tran} selects x by MRV with the fixed tie-break, and $\text{Erw}_\wedge$ expands the root using

$$\text{Div}(\text{Opt}(\varnothing, \{x, y, z\}), \text{Opt}(\{x \mapsto s_1\}, \{y, z\}), \text{Opt}(\{x \mapsto s_2\}, \{y, z\})).$$

These two child cases have lower bounds 0 and 4, respectively. They both belong to the selected division. The control must account for both, although it need not enumerate every completion of a pruned child. The depth-first f_{leaf} selects the $x \mapsto s_1$ child first.

Here $D_a(y) = \{s_2\}$ because x and y are noncompatible, whereas $D_a(z) = \{s_1, s_2\}$. MRV therefore selects y and creates the single legal child $y \mapsto s_2$. For $a = \{x \mapsto s_1, y \mapsto s_2\}$, the lower bound is 2, which is below the current $B = 5$. Both slots remain legal for z, giving two complete child cases with lower bounds 2 and 3.

Incumbent update and pruning. The following table records the recursive visit order after the root. A dash indicates that an offering is still unassigned.

Visit	x	y	z	LB	Action
1	s_1	–	–	0	Expand on y; only s_2 is legal.
2	s_1	s_2	–	2	Expand on z into both legal slots.
3	s_1	s_2	s_1	2	Complete and valid: replace the incumbent; $B \leftarrow 2$.
4	s_1	s_2	s_2	3	Complete and valid: score 3; keep the incumbent.
5	s_2	–	–	4	Prune: $4 \geq B = 2$; retain certificate 4.

At visit 3 the incumbent becomes $a_{\text{best}} = \{x \mapsto s_1, y \mapsto s_2, z \mapsto s_1\}$. Visit 4 is evaluated because the controller checks complete assignments before its lower-bound pruning rule. Its certificate is its exact score 3. After resolving the first root child, depth-first traversal reaches the $x \mapsto s_2$ child. Its preference penalty of 4 is already unavoidable, so no completion below it can improve the incumbent of score 2. The controller marks that case resolved with certificate 4 and generates no descendants there. Pruning does not declare it infeasible.

Combining results and stopping. The two z leaves return certificates 2 and 3. Their parent retains $\min(2, 3) = 2$; the single-child division on y also returns 2. The root then combines its two child certificates:

$$\ell_{\text{root}} = \min(2, 4) = 2 = B.$$

Every selected child case is now resolved, so the root can be marked yes and $G_\wedge$ holds. Resolved subtree detail may be contracted after preserving these certificates and the incumbent. The driver returns a_{best} with status OPTIMAL.

As an independent check, the four feasible assignments have scores

$(a(x), a(y), a(z))$	(s_1, s_2, s_1)	(s_1, s_2, s_2)	(s_2, s_1, s_1)	(s_2, s_1, s_2)
Eval(a)	2	3	4	5

and hence the minimum is indeed 2. The search did not need to enumerate the final two assignments during optimization: the retained bound accounted for their entire root child case.

7 End-to-End Integration and Comparison

7.1 Algorithm 4: Optional combined pipeline

Algorithm 3 Combined Three-Paradigm Pipeline

Require: Scheduling instance I
Ensure: Best valid timetable found, or FAILURE

```
a_0 ← ORFeasibility(I)
if a_0 = FAILURE then
    return FAILURE
end if
a_1 ← SetBasedImprove(I, a_0)
a_2 ← ANDOptimize(I, a_1)
return a_2                          ▷ Optimal only if AND stage completed
```

7.2 Comparison of the three paradigms

Table 2: Summary of proposed models

	OR-tree	Set-based	AND-tree
Purpose	Feasibility	Local improvement	Exact optimization
State	OR-tree	Assignment fact set	AND-tree
Primitive	*Altern*	*EXT*	*Div*
Control	DFS + MRV	Hill climbing	Branch-and-bound
Optimal?	Not targeted	Not guaranteed	If complete

[**TO FILL:** Explain each model's pros and cons: search memory, runtime, completeness, and solution quality. Avoid claiming an AND-tree guarantees optimality if the search budget expires.]

7.3 Unified worked example

[**TO FILL:** Use one small course-scheduling instance across all three models. Show the OR-tree's first feasible score, set-based's improved score, and AND-tree's exploration/proof of optimality. The same input and penalty parameters must be used in all three worked examples.]

8 Discussion and Limitations

The shared OR-tree stage may exhaust all alternatives without finding a feasible timetable; if it is interrupted before that determination, infeasibility has not been established, and neither proposal has a timetable to return. Set-based hill climbing can reach local minima because it only accepts improving Moves or Swaps, thus rejecting steps that could get it unstuck. AND-tree branch-and-bound preserves a complete feasible incumbent while covering the full assignment space, but its worst-case work is exponential; only a completed search or an incumbent with zero penalty establishes global optimality. AND-tree based search counts penalties into its lower bound that a later choice cannot undo, thus it never is able to skip the true best answer although pruning is only present deep in the search as very little is locked in at the start. For larger instances, set-based search is preferred because its predictable runtime suits a time budget, and the returned timetable should be labelled feasible rather than optimal when the AND-tree search is interrupted.

9 Conclusion

[**TO FILL:** Summarize the three models, their distinct state data structures, and why the proposed architecture is appropriate for the assignment. Do not claim experimentally verified performance before testing.]

We proposed two complete timetabling pipelines sharing an OR-tree feasibility stage. The set-based branch offers simple, feasibility-preserving local improvement, whereas AND-tree branch-and-bound searches across alternative assignments and can prove an optimum when completed. Both use the same hard-constraint checker and evaluation objective. The proposed models and worked derivations specify how each search process can be instantiated for the course timetabling problem.

References

[1] CPSC 433. *Project Description*, Version 1, University of Calgary, Fall 2025.

[2] CPSC 433. *Project Problem Description*, Version 1, University of Calgary, Fall 2025.

[3] CPSC 433. *Lecture 05: Search*, University of Calgary.

[4] CPSC 433. *Lecture 06: Set-Based Search*, University of Calgary.

[5] CPSC 433. *Lecture 09: And-Tree-Based Search*, University of Calgary.

[6] CPSC 433. *Lecture 11: Or-Tree-Based Search*, University of Calgary.

[7] CPSC 433. *Lecture 14: Search Controls*, University of Calgary.

A Complete Constraint Formulas

[**TO FILL:** Optional: extended detailed formalization of all hard and soft constraints.]

B Additional Derivations and Search Trees

[**TO FILL:** Optional: extra OR-tree, AND-tree, and set-based state traces.]